

Contents

- Programming 1 - Interference Removal using Frequency Domain
- (2) Provide a detailed explanation of how your program/algorithm works.
- (3) How do you identify the “most out of place” frequency in the Fourier domain?
- (4) Why is it not sufficient to simply zero out the unwanted frequency component?
- (5) How can you reconstruct the image after modifying its frequency domain representation?
- (6) What are common filtering techniques to suppress unwanted frequencies while minimizing image distortion?
- (7) How do you verify if the interference removal was successful?
- Programming 2 - Pseudocolor using Intensity Slicing
- (2) Provide a detailed explanation of how your program/algorithm works.
- (3) What is intensity slicing, and how does it help in generating pseudo-color images?
- (4) How do you determine appropriate intensity ranges for mapping grayscale values to the eight colors?
- (5) How can you ensure that your pseudo-color mapping does not distort the original image details?
- (6) How can you evaluate the effectiveness of your pseudo-color transformation?
- Programming 3 - Image Segmentation using Different Color Spaces
- (2) Provide a detailed explanation of how your program/algorithm works.
- (3) Describe the segmentation methods used for each color space
- (4) Which of the color spaces seems to perform better in segmenting this image?
- (5) What is your reasoning for this?

```
% Problem 1: Periodic Interference Removal
clc; clear; close all;

% Step 1: Read image
img = imread('interfere.pgm');
img = im2double(img);

% Step 2: FFT
F = fft2(img);
F_shift = fftshift(F);

% Step 3: Magnitude (for visualization)
magnitude = log(1 + abs(F_shift));

figure;
imshow(magnitude, []);
title('Fourier Magnitude Spectrum');

% Step 4: Find peaks (interference frequency)
% Threshold to detect bright spikes
threshold = max(magnitude(:)) * 0.7;
[rows, cols] = find(magnitude > threshold);

% Remove center (DC component)
center = [size(img,1)/2, size(img,2)/2];
dist = sqrt((rows - center(1)).^2 + (cols - center(2)).^2);

% Ignore very close to center
```

```

valid_idx = dist > 20;
rows = rows(valid_idx);
cols = cols(valid_idx);

% Step 5: Create notch filter (smooth removal)
H = ones(size(img));

D0 = 10; % radius of notch

for k = 1:length(rows)
    u = rows(k);
    v = cols(k);

    for i = 1:size(img,1)
        for j = 1:size(img,2)

            D1 = sqrt((i-u)^2 + (j-v)^2);
            D2 = sqrt((i-(2*center(1)-u))^2 + (j-(2*center(2)-v))^2);

            % Butterworth notch filter
            H(i,j) = H(i,j) * ...
                (1 / (1 + (D0/D1)^4)) * ...
                (1 / (1 + (D0/D2)^4));

        end
    end
end

% Step 6: Apply filter
F_filtered = F_shift .* H;

% Step 7: Inverse FFT
F_ishift = ifftshift(F_filtered);
img_clean = real(ifft2(F_ishift));

% Filtered Spectrum
figure;
imshow(log(1 + abs(F_filtered)), []);
title('Filtered Spectrum');

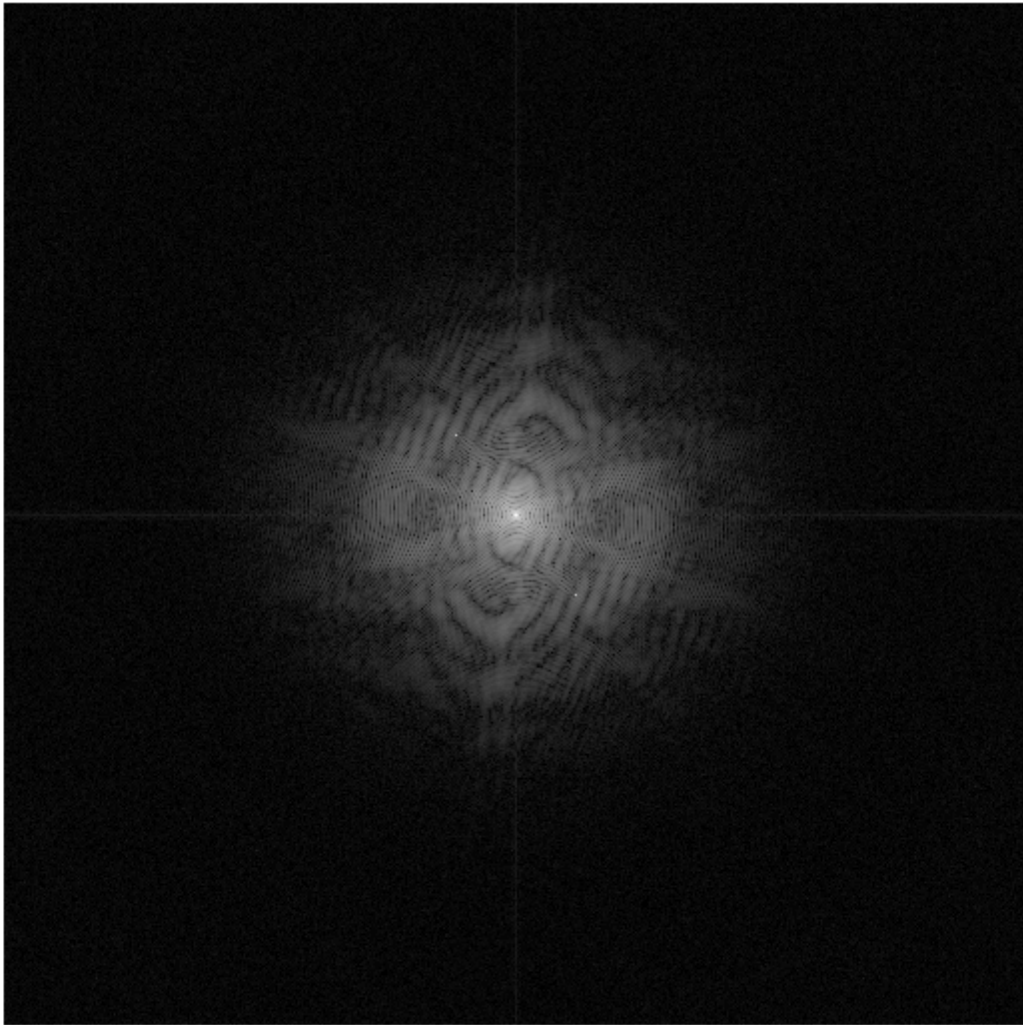
% Difference Image
figure;
imshow(abs(img - img_clean), []);
title('Difference Image');

% Step 8: Show results
figure;
subplot(1,2,1);
imshow(img, []);
title('Original Image');

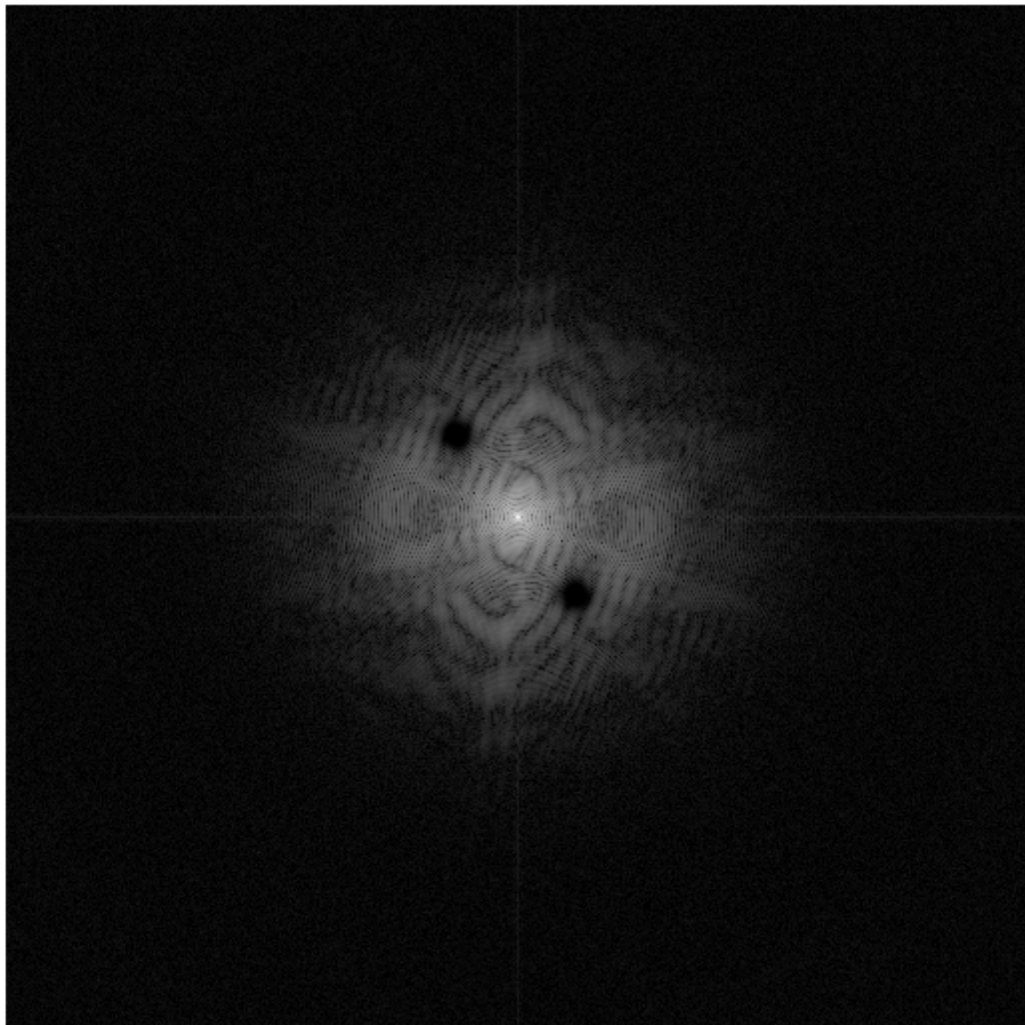
subplot(1,2,2);
imshow(img_clean, []);
title('Filtered Image');

```

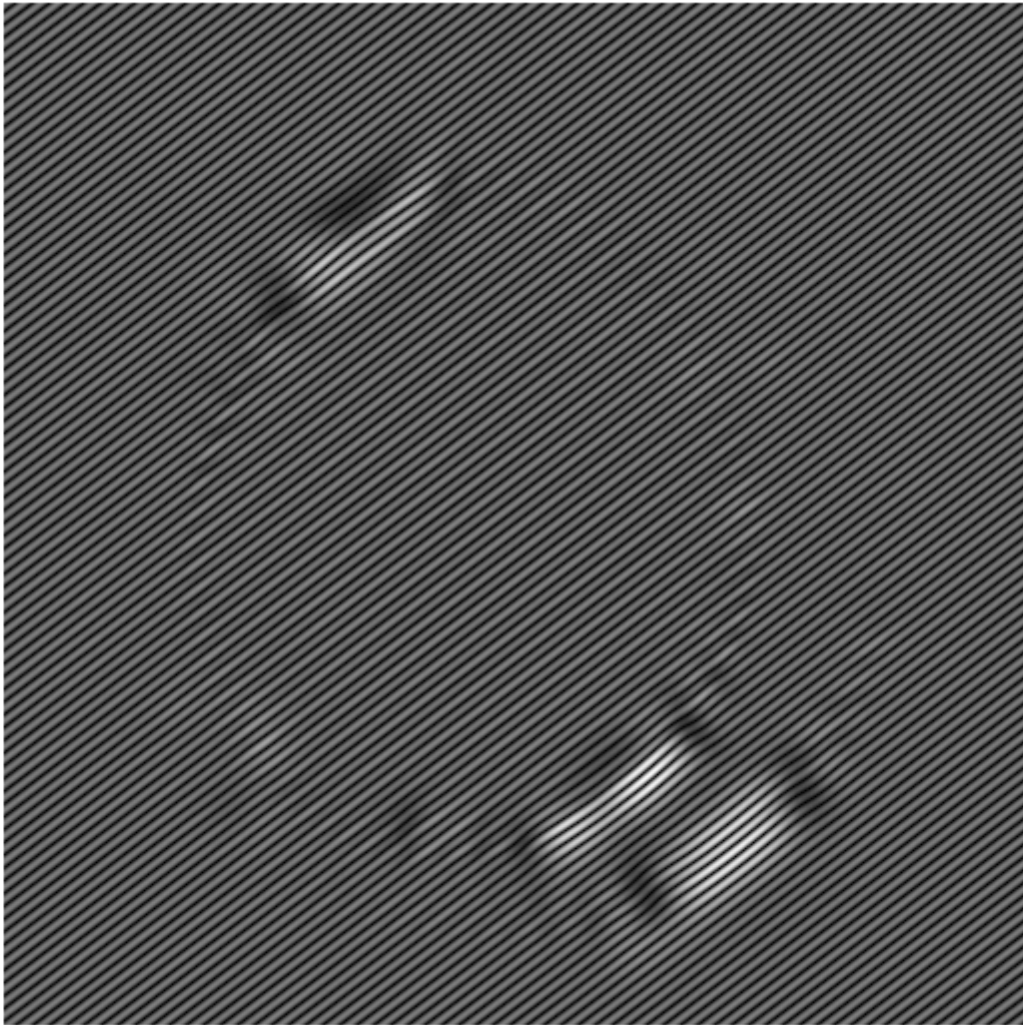
Fourier Magnitude Spectrum

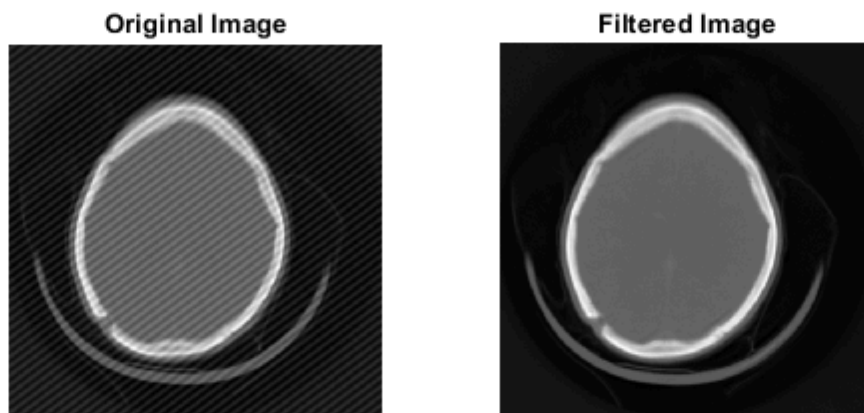


Filtered Spectrum



Difference Image





Programming 1 - Interference Removal using Frequency Domain

(2) Provide a detailed explanation of how your program/algorithm works.

```
% First, the input image is converted into the frequency domain using FFT.  
% In the Fourier domain, periodic noise appears as bright spikes at certain locations.  
  
% The algorithm automatically detects these spikes by finding values that  
% are significantly higher than the surrounding frequencies.  
  
% Instead of directly removing these frequencies, a smooth notch filter  
% (Butterworth filter) is applied around those spike locations.  
  
% Finally, the modified frequency representation is converted back to the  
% spatial domain using inverse FFT to obtain the cleaned image.
```

(3) How do you identify the “most out of place” frequency in the Fourier domain?

```
% The unwanted frequency is not the strongest one (center), but the one  
% that appears as bright isolated spikes away from the center.  
  
% These spikes are "out of place" because natural images usually have  
% most of their energy concentrated near the center.  
  
% By setting a threshold and ignoring the center region, these spikes  
% can be automatically detected.
```

(4) Why is it not sufficient to simply zero out the unwanted frequency component?

```
% Directly setting the frequency to zero creates sharp discontinuities
% in the frequency domain.

% This leads to artifacts such as ringing or distortion in the spatial domain.

% Instead, a smooth filter is used so that the transition is gradual,
% which helps preserve the quality of the image.
```

(5) How can you reconstruct the image after modifying its frequency domain representation?

```
% After modifying the frequency domain, the inverse Fourier Transform (IFFT)
% is applied.

% This converts the filtered frequency representation back into the
% spatial domain.

% The result is an image where the periodic noise is reduced while
% maintaining important structures.
```

(6) What are common filtering techniques to suppress unwanted frequencies while minimizing image distortion?

```
% Some commonly used techniques to remove unwanted frequencies include:

% 1. Notch Filters - remove specific frequencies (used in this project)
% 2. Low-pass Filters - remove high-frequency noise
% 3. High-pass Filters - remove low-frequency components
% 4. Butterworth Filters - smooth transition, less distortion
% 5. Gaussian Filters - very smooth and natural filtering
```

(7) How do you verify if the interference removal was successful?

```
% The effectiveness of the interference removal can be verified in three ways:

% 1. Visual Inspection:
%   The stripe pattern in the original image is reduced in the filtered image.

% 2. Frequency Domain Check:
%   The bright spikes seen in the Fourier spectrum disappear after filtering.

% 3. Difference Image:
%   The difference between original and filtered image mainly shows the
%   removed stripe pattern, confirming that only noise was removed.

% Overall, the results show that the interference has been successfully removed
% without significantly affecting the important image details.
```

Programming 2 - Pseudocolor using Intensity Slicing

```
clc; clear; close all;
```

```

img = imread('swan.pgm');

if size(img,3) == 3
    img = rgb2gray(img);
end

img = im2uint8(img);

[M, N] = size(img);
color_img = zeros(M, N, 3, 'uint8');

levels = linspace(0, 256, 9);

for i = 1:M
    for j = 1:N
        val = img(i,j);

        if val < levels(2)
            color = [0 0 0];
        elseif val < levels(3)
            color = [0 0 255];
        elseif val < levels(4)
            color = [0 255 0];
        elseif val < levels(5)
            color = [255 255 0];
        elseif val < levels(6)
            color = [255 165 0];
        elseif val < levels(7)
            color = [128 0 128];
        elseif val < levels(8)
            color = [165 42 42];
        else
            color = [255 0 0];
        end

        color_img(i,j,:) = color;
    end
end

figure;
subplot(1,2,1);
imshow(img);
title('Swan - Grayscale');

subplot(1,2,2);
imshow(color_img);
title('Swan - Pseudocolor');

img = imread('tiger.jpeg');

if size(img,3) == 3
    img = rgb2gray(img);
end

img = im2uint8(img);

```

```
[M, N] = size(img);
color_img = zeros(M, N, 3, 'uint8');

for i = 1:M
    for j = 1:N
        val = img(i,j);

        if val < levels(2)
            color = [0 0 0];
        elseif val < levels(3)
            color = [0 0 255];
        elseif val < levels(4)
            color = [0 255 0];
        elseif val < levels(5)
            color = [255 255 0];
        elseif val < levels(6)
            color = [255 165 0];
        elseif val < levels(7)
            color = [128 0 128];
        elseif val < levels(8)
            color = [165 42 42];
        else
            color = [255 0 0];
        end

        color_img(i,j,:) = color;
    end
end

figure;
subplot(1,2,1);
imshow(img);
title('Tiger - Grayscale');

subplot(1,2,2);
imshow(color_img);
title('Tiger - Pseudocolor');
```

Swan - Grayscale



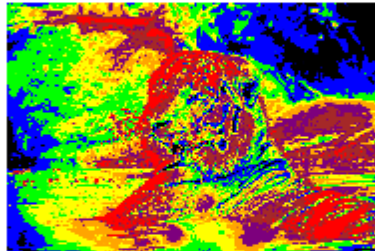
Swan - Pseudocolor



Tiger - Grayscale



Tiger - Pseudocolor



(2) Provide a detailed explanation of how your program/algorithm works.

First, the input image is read and converted to grayscale if it is not already. The grayscale image contains intensity values ranging from 0 to 255.

```
% The intensity range is divided into 8 equal intervals using linspace.

% For each pixel, its intensity value is checked to determine which range
% it belongs to.

% Based on the range, a specific RGB color is assigned to that pixel.

% This process is repeated for all pixels, resulting in a pseudocolor image
% where different intensity levels are represented using different colors.
```

(3) What is intensity slicing, and how does it help in generating pseudo-color images?

```
% Intensity slicing is a technique where the grayscale intensity values
% are divided into multiple ranges (or slices).

% Each range is assigned a specific color instead of a grayscale value.

% This helps in highlighting different regions of the image that may not
% be easily distinguishable in grayscale.

% In this project, the grayscale image is divided into 8 slices, each
% mapped to a different color.
```

(4) How do you determine appropriate intensity ranges for mapping grayscale values to the eight colors?

```
% The intensity range (0-255) is divided into 8 equal parts using linspace.

% This ensures that each color represents an equal portion of the intensity range.

% Equal spacing is simple and works well for general images.

% In some cases, ranges can also be adjusted based on image histogram
% to highlight specific features more clearly.
```

(5) How can you ensure that your pseudo-color mapping does not distort the original image details?

```
% The pseudocolor mapping does not change the structure of the image.

% It only changes how intensity values are visualized using colors.

% Since each pixel is mapped directly based on its intensity,
% spatial details and edges are preserved.

% No filtering or smoothing is applied, so the original information remains intact.
```

(6) How can you evaluate the effectiveness of your pseudo-color transformation?

```
% The effectiveness of pseudocolor transformation can be evaluated by:

% 1. Visual Improvement:
%    Different regions become easier to distinguish compared to grayscale.
```

```
% 2. Contrast Enhancement:
%   Subtle intensity differences become more noticeable using colors.

% 3. Comparison Across Images:
%   The tiger image shows more variation in colors due to its complex texture,
%   while the swan image has smoother regions.

% Overall, pseudocoloring improves interpretability without modifying
% the original image content.
```

Programming 3 - Image Segmentation using Different Color Spaces

```
clc; clear; close all;

% Step 1: Read image
img = imread('scene.ppm');
img = im2double(img);

% RGB
rgb_data = reshape(img, [], 3);

idx_rgb = kmeans(rgb_data, 3);

seg_rgb = reshape(idx_rgb, size(img,1), size(img,2));

figure;
imshow(label2rgb(seg_rgb));
title('RGB Segmentation');

% HSI
hsv_img = rgb2hsv(img);
hsv_data = reshape(hsv_img, [], 3);

idx_hsv = kmeans(hsv_data, 3);

seg_hsv = reshape(idx_hsv, size(img,1), size(img,2));

figure;
imshow(label2rgb(seg_hsv));
title('HSV Segmentation');

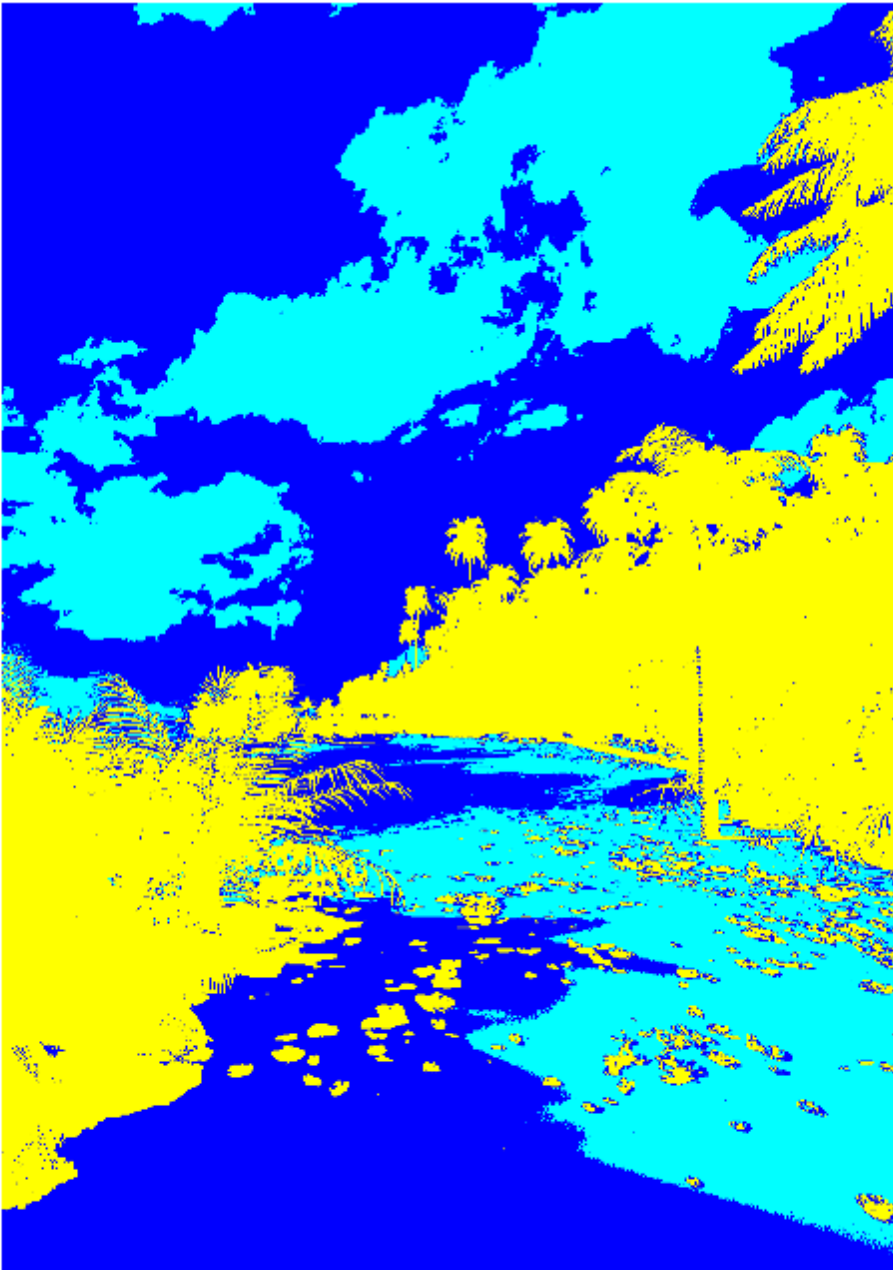
% LAB
lab_img = rgb2lab(img);
lab_data = reshape(lab_img, [], 3);

idx_lab = kmeans(lab_data, 3);

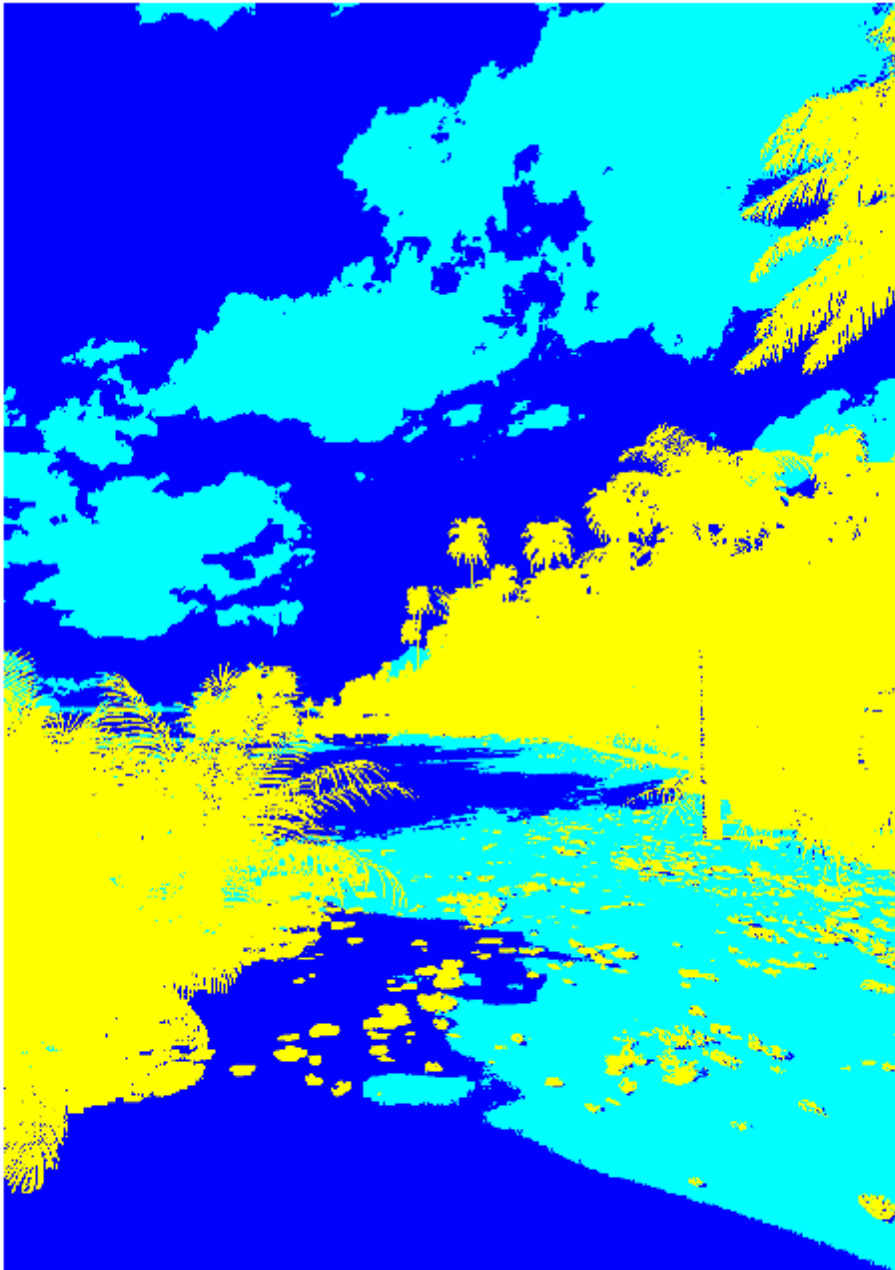
seg_lab = reshape(idx_lab, size(img,1), size(img,2));

figure;
imshow(label2rgb(seg_lab));
title('LAB Segmentation');
```

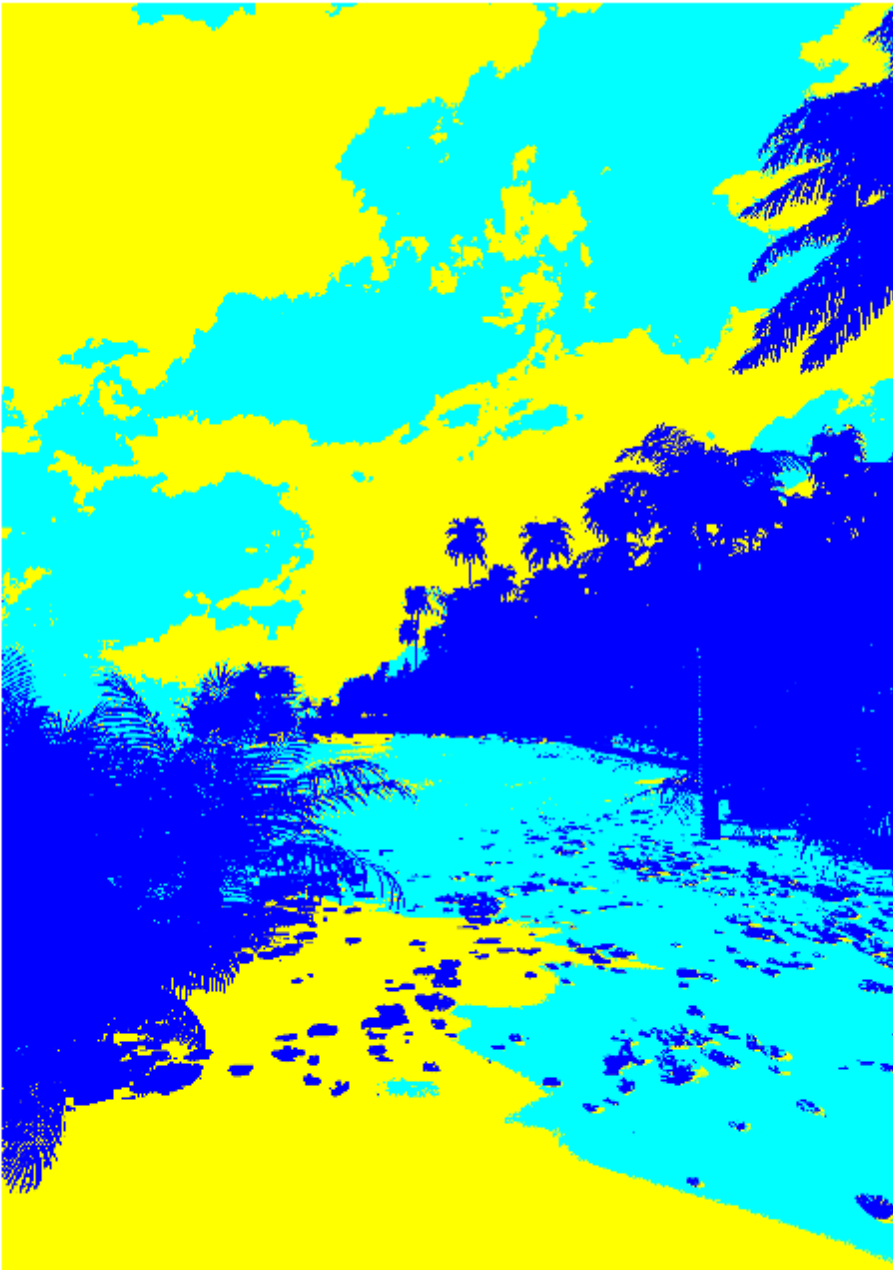
RGB Segmentation



HSV Segmentation



LAB Segmentation



(2) Provide a detailed explanation of how your program/algorithm works.

```
% First, the input image is read and converted into double format.
```

```
% The image is then represented in three different color spaces:  
% RGB (original), HSV (used as HSI), and LAB.
```

```
% For each color space, the image is reshaped into a 2D array where  
% each pixel is treated as a data point with three values.
```

```
% K-means clustering is applied to group pixels into 3 clusters  
% based on similarity in color values.
```

```
% The cluster labels are reshaped back into image form to produce  
% segmented images.
```

(3) Describe the segmentation methods used for each color space

```
% RGB:  
% Segmentation is performed using the red, green, and blue intensity values.  
% Since RGB combines color and brightness, it is sensitive to lighting changes,  
% which may lead to less accurate segmentation.  
  
% HSV (HSI):  
% The image is converted to HSV, where hue represents color and value  
% represents brightness.  
% This separation helps group pixels with similar colors even if lighting varies,  
% leading to improved segmentation compared to RGB.  
  
% LAB:  
% The LAB color space separates lightness (L) from color components (A and B).  
% It is designed to reflect human perception of color differences.  
% This allows better grouping of visually similar regions.
```

(4) Which of the color spaces seems to perform better in segmenting this image?

```
% The LAB color space performs the best for segmenting this image.  
  
% The segmented output shows clearer and more uniform regions compared  
% to RGB and HSV.
```

(5) What is your reasoning for this?

```
% LAB performs better because it separates brightness from color information.  
  
% Unlike RGB, it is less affected by lighting variations.  
  
% Compared to HSV, LAB represents color differences more accurately  
% based on human perception.  
  
% As a result, pixels with similar colors are grouped more consistently,  
% leading to better segmentation quality.
```
